

Transformer模型学习笔记



一、简介

Google于2017年6月发布在arxiv上的一篇文章《Attention is all you need》，提出解决sequence to sequence问题的transformer模型，用全attention的结构代替了lstm，抛弃了之前传统的encoder-decoder模型必须结合cnn或者rnn的固有模式

二、 encoder-decoder模型

所谓encoder-decoder模型，又叫做编码-解码模型。这是一种应用于seq2seq问题的模型。seq2seq指根据一个输入序列x来生成输出序列y，比如翻译的场景，从一个语言的句子翻译成另外一个语言的句子。

为了解决seq2seq问题，有人提出了encoder-decoder模型，也就是编码-解码模型。所谓编码，就是将输入序列转化为一个固定长度的向量；解码，就是将之前生成的固定向量再转化成输出序列。

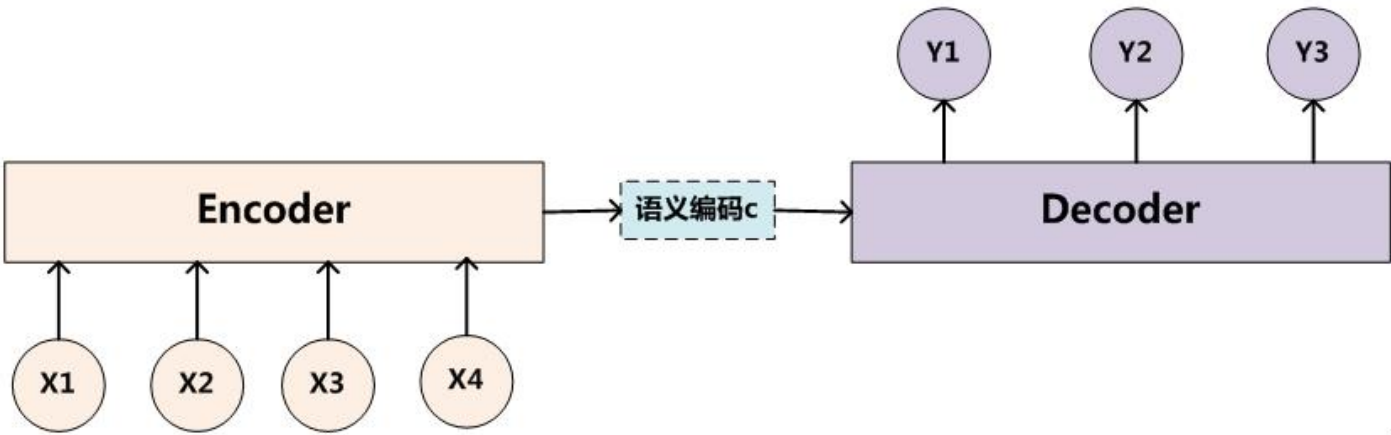
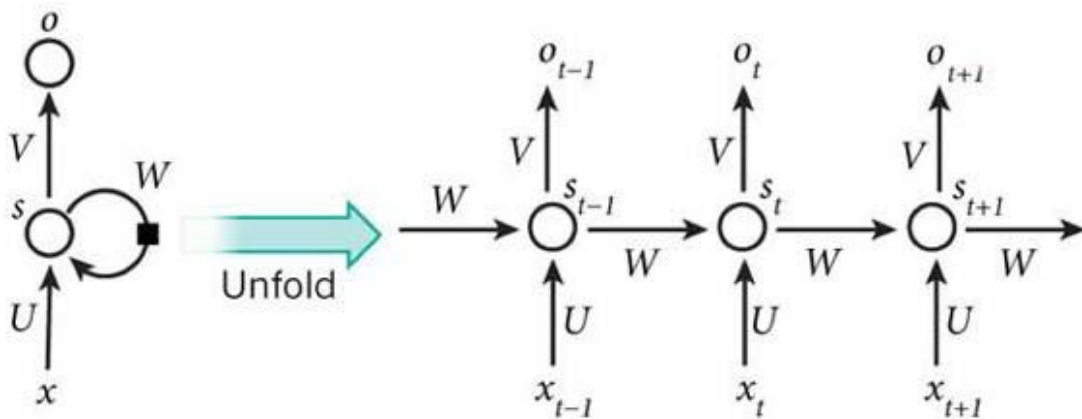


图1 encoder-decoder图解

在机器翻译例子中，Encoder和decoder一般选取RNN或者LSTM。



$$O_t = g(V \cdot S_t)$$

$$S_t = f(U \cdot X_t + W \cdot S_{t-1})$$

S_t 的值不仅仅取决于 X_t ，还取决于 S_{t-1}

图2 rnn图解

三、RNN方式encoder-decoder缺陷

RNN（或者LSTM，GRU等）的计算限制为是顺序的，也就是说RNN相关算法只能从左向右依次计算或者从右向左依次计算，这种机制带来了两个问题：

- 顺序计算的过程中信息会丢失，尽管LSTM等门机制的结构一定程度上缓解了长期依赖的问题，但是对于特别长期的依赖现象,LSTM依旧无能为力；
- 时间片 t 的计算依赖 $t-1$ 时刻的计算结果，这样限制了模型的并行能力。

四、Attention机制

视觉注意力机制是人类视觉所特有的大脑信号处理机制。人类视觉通过快速扫描全局图像，获得需要重点关注的目标区域，也就是一般所说的注意力焦点，而后对这一区域投入更多注意力资源，以获取更多所需要关注目标的细节信息，而抑制其他无用信息。

机器翻译中的注意力机制指在生成翻译结果的第 i 个输出时，我们应该主要关注原句子的哪些词。

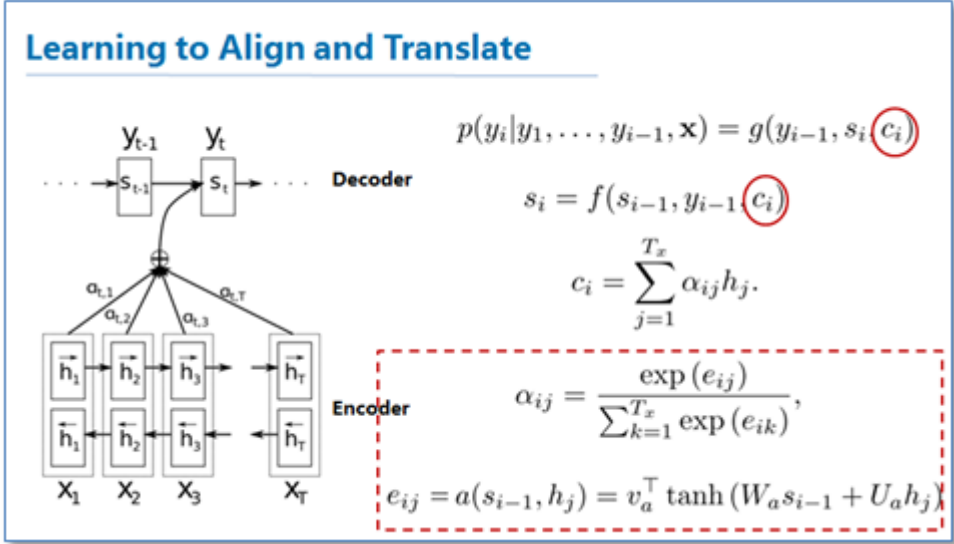
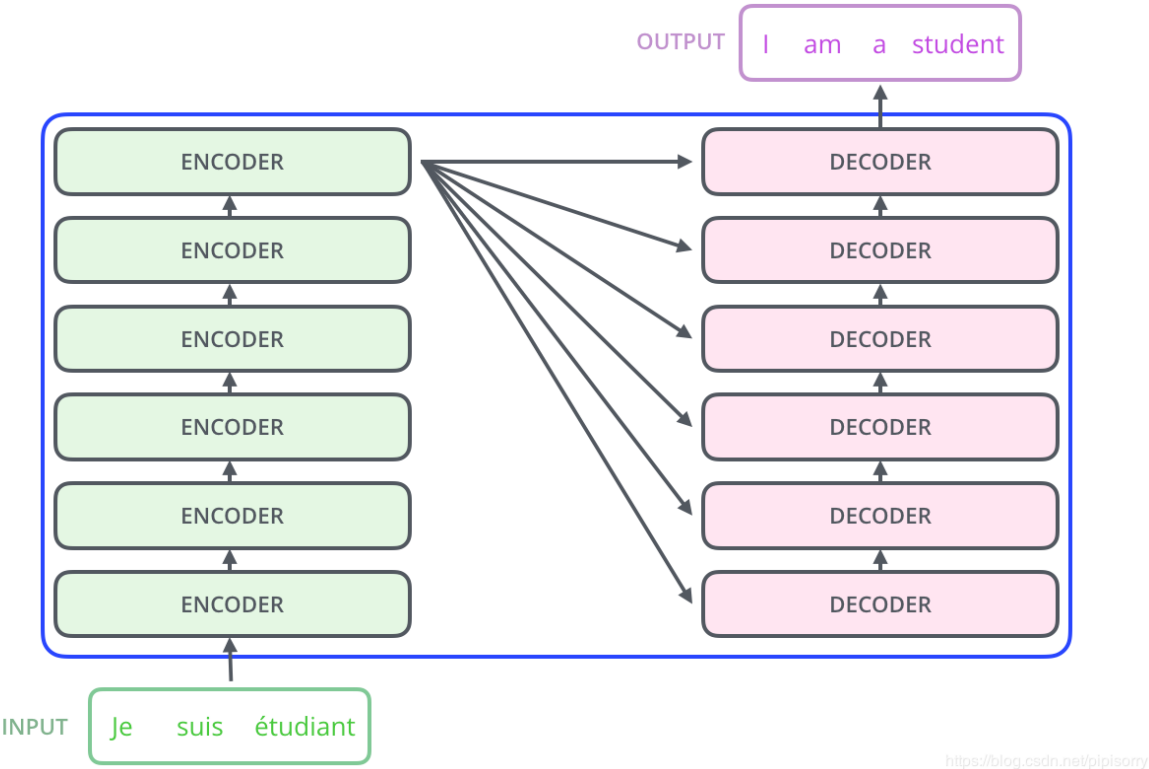


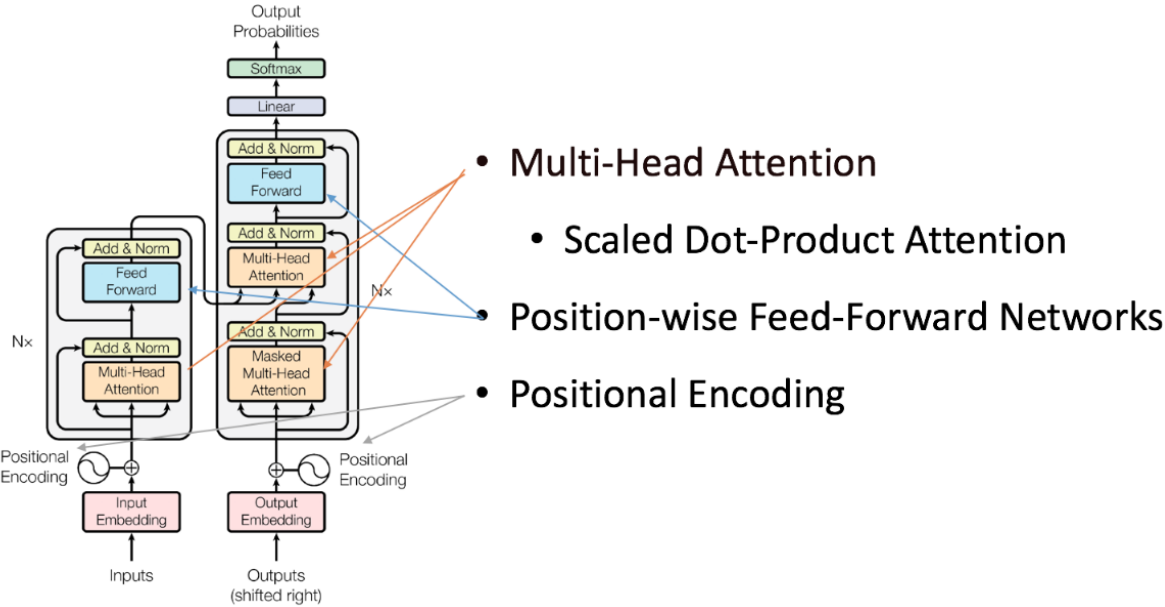
图3 注意力图解

五、transformer模型

抛弃了之前传统的encoder-decoder模型必须结合cnn或者rnn的固有模式，只用attention，解决sequence to sequence问题。

5.1 transformer结构





<https://blog.csdn.net/pipisorry>

图4 transformer结构图解

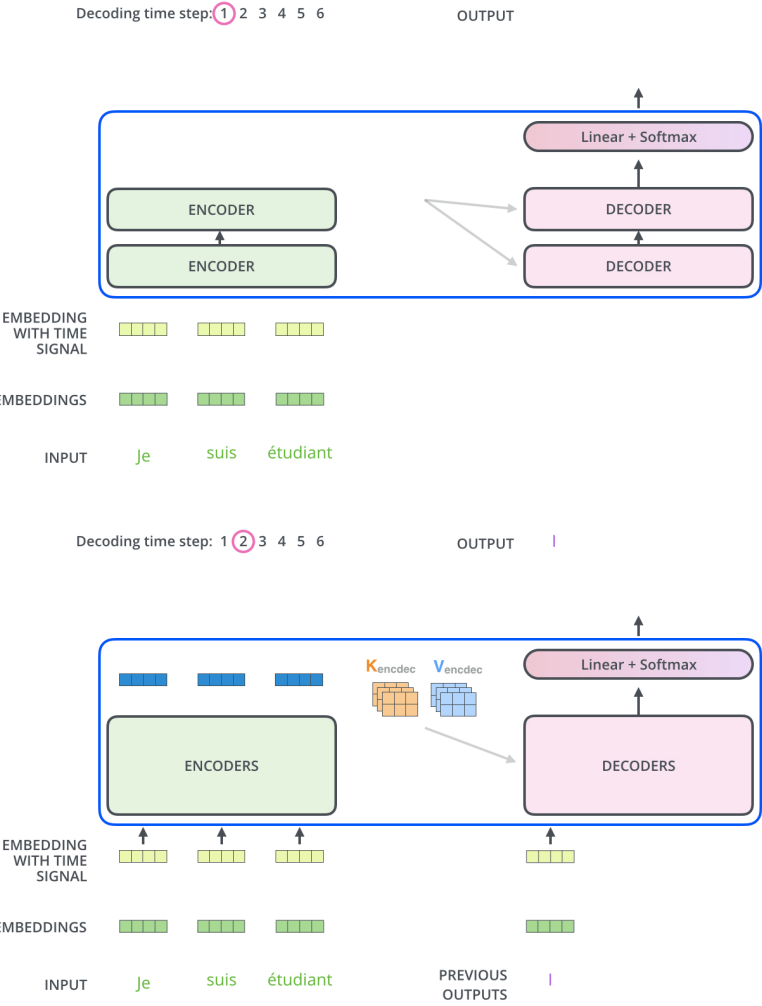
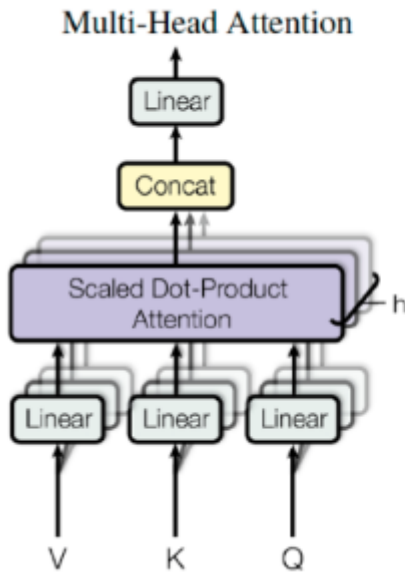


图 5 tranformer工作流程图解

5.2 multi-head attention



论文提出对queries, keys和values做h次不同的投影, 映射的维度都是 d^k 和 d^v , 然后都经过 Scaled Dot-Product Attention, 将结果拼接在一起, 最后通过一个线性映射输出, 通过多头注意力, 模型能够获得不同子空间下的位置信息。如上图右边所示, 公式如下:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Transformer会在三个地方使用multi-head attention:

- **encoder-decoder attention**: 输入为encoder的输出和decoder的self-attention输出, 其中encoder的self-attention作为 key and value, decoder的self-attention作为query。
- **encoder self-attention**: 输入的Q、K、V都是encoder的input embedding and positional embedding。
- **decoder self-attention**: 在decoder的self-attention层中, decoder 都能够访问当前位置前面的位置, 输入的Q、K、V都是decoder的input embedding and positional embedding。

5.3 position-wise feed-forward networks

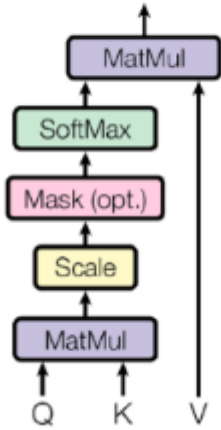
经过全连接计算后, 输入输出维度一样

$$\text{FFN}(Z) = \max(0, ZW_1 + b_1)W_2 + b_2$$

5.4 masked multi-head attention

对于decoder中的第一个多头注意力子层，需要添加masking，确保预测位置*i*的时候仅仅依赖于位置小于*i*的输出，确保预测第*i*个位置时不会接触到未来的信息。在机器翻译中，解码过程是一个顺序操作的过程，也就是当解码第*i*个特征向量时，我们只能看到第*i*-1及其之前的解码结果。而我们要计算的attention是根据当前词*i*和前面的词来的，不能是后面的。

Scaled Dot-Product Attention



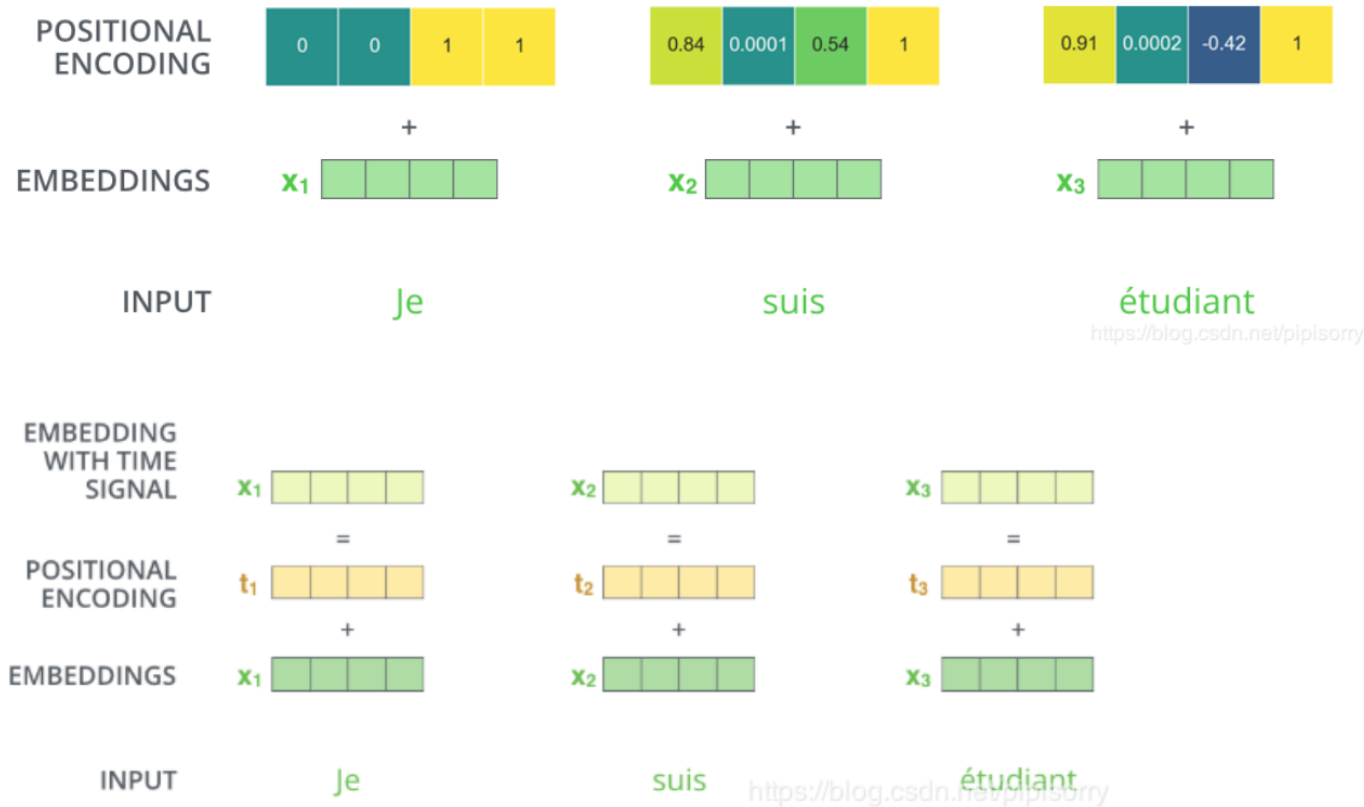
5.5 position encoding

Transformer抛弃了RNN，而RNN最大的优点就是在时间序列上对数据的抽象，所以文章中作者提出两种Positional Encoding的方法，将Positional Encoding后的数据与输入embedding数据求和，加入了相对位置信息。

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Note: *pos* 表示单词的位置，*i* 表示单词的维度，*d_model*表示embedding维度512。关于位置编码的实现可在Google开源的算法中[get_timing_signal_1d\(\)](#)函数找到对应的代码。



position编码示例

position编码涉及到三角函数计算，由于编码维度和单词位置确定后，position编码也确定，该编码可以提前计算好

5.6 损失层

解码器解码之后，解码的特征向量经过一层激活函数为softmax的全连接层之后得到反映每个单词概率的输出向量。此时我们便可以通过CTC等损失函数训练模型了

5.7 evaluation

《Attention is All You Need》论文用8张P100的GPU进行训练，英德翻译 base model（参数：65*10^6）训练12小时， big model(参数：213*10^6) 3.5天
准确性从目前最好的26.36 提高到 28.4，后者总计算量为老系统的1/3。

六、 paddle训练结果

6.1 transformer配置

```
GPU p40

python -u train.py --batch_size 3200 --pool_size 200000 n_layer 6 n_head 16 d_model 1024
d_inner_hid 4096 n_head 16 prepostprocess_dropout 0.3
```

6.2 统计结果

1		words per step	step latency(s)	word throughput (word / s)
2	单卡	3200	0.877	3649
3	双卡	6400	1.0	6400

耗时占比前3的op （训练了10个step）

1		op	radio	调用次数
2	单卡	read	22%	960
3		mul_grad	21%	620
4		dropout	14%	960
5	双卡	all_reduce	44%	3700
6		mul_grad	11%	1920
7		read	8%	40

6.3 单卡训练原始数据

Event	Calls	Total	Min.	Max.	Ave.	Ratio.
read	20	2612.12	0.002048	1305.84	130.606	0.224014
mul_grad	960	2494.88	1.25645	6.0631	2.59883	0.21396
dropout	620	1653.48	0.807936	7.70662	2.6669	0.141801
mul	960	1256.04	0.60416	5.86752	1.30837	0.107717
matmul_grad	370	616.566	0.6144	4.98893	1.66639	0.0528764
elementwise_add_grad	740	404.278	0.062464	80.726	0.546322	0.0346707
matmul	370	399.732	0.333824	2.87232	1.08036	0.0342808
lookup_table	40	234.649	0.113664	152.38	5.86621	0.0201233
adam	1850	224.974	0.008192	0.497664	0.121607	0.0192936
layer_norm_grad	320	212.645	0.551936	0.708608	0.664515	0.0182363
dropout_grad	620	186.377	0.1024	0.722944	0.300608	0.0159836
elementwise_add	740	175.877	0.101376	0.512	0.237672	0.0150831

layer_norm	320	161.386	0.442368	0.600064	0.504333	0.0138404
sum	440	119.136	0.157696	1.98144	0.270764	0.0102171
softmax	180	118.039	0.602112	0.761856	0.65577	0.0101229
transpose2	720	115.296	0.141312	0.18944	0.160134	0.00988775
transpose2_grad	720	114.365	0.141312	0.166912	0.158841	0.00980792
softmax_grad	180	107.441	0.560128	0.667648	0.596895	0.0092141
elementwise_pow	10	89.855	0.114688	28.4682	8.9855	0.00770592
relu_grad	120	86.5792	0.633856	0.741376	0.721493	0.00742499
scale	4100	81.7962	0.008192	0.182272	0.0199503	0.0070148
fill_zeros_like	1260	62.6831	0.008192	0.216064	0.0497485	0.00537567
relu	120	60.1027	0.442368	0.516096	0.500855	0.00515437
elementwise_mul	40	36.5619	0.008192	31.8126	0.914048	0.00313553
softmax_with_cross_entropy	10	12.2583	1.20218	1.24928	1.22583	0.00105127
lookup_table_grad	20	5.56646	0.256	0.3072	0.278323	0.000477377
softmax_with_cross_entropy_grad	10	5.54598	0.544768	0.559104	0.554598	0.000475621
label_smooth	10	4.32538	0.422912	0.468992	0.432538	0.000370942
reshape2_grad	730	2.07053	0.002048	0.006144	0.00283634	0.000177567
reshape2	730	2.03056	0.002016	0.004096	0.00278159	0.00017414
one_hot	10	1.96819	0.188416	0.238624	0.196819	0.000168791
fill_constant	40	0.777216	0.008192	0.12288	0.0194304	6.66536e-05
reduce_sum	20	0.219136	0.01024	0.012288	0.0109568	1.8793e-05
increment	10	0.144384	0.002048	0.06656	0.0144384	1.23823e-05
reduce_sum_grad	10	0.141312	0.013312	0.014336	0.0141312	1.21188e-05
cast	10	0.126976	0.002048	0.07168	0.0126976	1.08894e-05
create_double_buffer_reader	10	0.125952	0.002048	0.088064	0.0125952	1.08016e-05
elementwise_min	10	0.09216	0.009216	0.009216	0.009216	7.90359e-06
elementwise_div	10	0.09216	0.008192	0.01024	0.009216	7.90359e-06
elementwise_div_grad	10	0.090112	0.008192	0.009216	0.0090112	7.72796e-06
elementwise_mul_grad	10	0.089088	0.008192	0.009216	0.0089088	

7.64014e-06

ScopeBufferedSSAGraphExecutorAfterRun 10 0 0 0 0 0

[2019-01-08 11:00:29,438 INFO train.py:550] step_idx: 100, epoch: 0, batch: 100, avg loss: 6.599910, normalized loss: 5.454963, ppl: 735.028870, speed: 1.12 step/s

[2019-01-08 11:01:57,666 INFO train.py:550] step_idx: 200, epoch: 0, batch: 200, avg loss: 5.737191, normalized loss: 4.592244, ppl: 310.191772, speed: 1.13 step/s

[2019-01-08 11:03:25,492 INFO train.py:550] step_idx: 300, epoch: 0, batch: 300, avg loss: 5.557585, normalized loss: 4.412638, ppl: 259.196198, speed: 1.14 step/s

[2019-01-08 11:04:52,912 INFO train.py:550] step_idx: 400, epoch: 0, batch: 400, avg loss: 5.500680, normalized loss: 4.355734, ppl: 244.858490, speed: 1.14 step/s

[2019-01-08 11:06:20,716 INFO train.py:550] step_idx: 500, epoch: 0, batch: 500, avg loss: 5.130758, normalized loss: 3.985811, ppl: 169.145325, speed: 1.14 step/s

[2019-01-08 11:07:48,796 INFO train.py:550] step_idx: 600, epoch: 0, batch: 600, avg loss: 4.505666, normalized loss: 3.360719, ppl: 90.528595, speed: 1.14 step/s

[2019-01-08 11:09:17,433 INFO train.py:550] step_idx: 700, epoch: 0, batch: 700, avg loss: 4.364899, normalized loss: 3.219952, ppl: 78.641472, speed: 1.13 step/s

[2019-01-08 11:10:45,800 INFO train.py:550] step_idx: 800, epoch: 0, batch: 800, avg loss: 4.198206, normalized loss: 3.053259, ppl: 66.566803, speed: 1.13 step/s

[2019-01-08 11:12:13,406 INFO train.py:550] step_idx: 900, epoch: 0, batch: 900, avg loss: 4.161342, normalized loss: 3.016395, ppl: 64.157539, speed: 1.14 step/s

[2019-01-08 11:13:41,474 INFO train.py:550] step_idx: 1000, epoch: 0, batch: 1000, avg loss: 4.298094, normalized loss: 3.153147, ppl: 73.559479, speed: 1.14 step/s

[2019-01-08 11:15:10,433 INFO train.py:550] step_idx: 1100, epoch: 0, batch: 1100, avg loss: 4.241940, normalized loss: 3.096994, ppl: 69.542671, speed: 1.12 step/s

[2019-01-08 11:16:38,167 INFO train.py:550] step_idx: 1200, epoch: 0, batch: 1200, avg loss: 4.093373, normalized loss: 2.948426, ppl: 59.941723, speed: 1.14 step/s

[2019-01-08 11:18:06,124 INFO train.py:550] step_idx: 1300, epoch: 0, batch: 1300, avg loss: 4.129523, normalized loss: 2.984576, ppl: 62.148289, speed: 1.14 step/s

[2019-01-08 11:19:33,939 INFO train.py:550] step_idx: 1400, epoch: 0, batch: 1400, avg loss: 4.021961, normalized loss: 2.877014, ppl: 55.810455, speed: 1.14 step/s

[2019-01-08 11:21:01,367 INFO train.py:550] step_idx: 1500, epoch: 0, batch: 1500, avg loss: 4.111493, normalized loss: 2.966546, ppl: 61.037758, speed: 1.14 step/s

[2019-01-08 11:22:28,872 INFO train.py:550] step_idx: 1600, epoch: 0, batch: 1600, avg loss: 3.798538, normalized loss: 2.653591, ppl: 44.635880, speed: 1.14 step/s

[2019-01-08 11:23:56,976 INFO train.py:550] step_idx: 1700, epoch: 0, batch: 1700, avg loss: 3.703383, normalized loss: 2.558436, ppl: 40.584370, speed: 1.14 step/s

[2019-01-08 11:25:25,376 INFO train.py:550] step_idx: 1800, epoch: 0, batch: 1800, avg loss: 3.547788, normalized loss: 2.402841, ppl: 34.736393, speed: 1.13 step/s

[2019-01-08 11:26:54,106 INFO train.py:550] step_idx: 1900, epoch: 0, batch: 1900, avg loss: 3.521598, normalized loss: 2.376651, ppl: 33.838470, speed: 1.13 step/s

[2019-01-08 11:28:21,931 INFO train.py:550] step_idx: 2000, epoch: 0, batch: 2000, avg loss: 3.508376, normalized loss: 2.363429, ppl: 33.393990, speed: 1.14 step/s

[2019-01-08 11:29:49,457 INFO train.py:550] step_idx: 2100, epoch: 0, batch: 2100, avg loss: 3.508539, normalized loss: 2.363592, ppl: 33.399426, speed: 1.14 step/s

[2019-01-08 11:31:17,058 INFO train.py:550] step_idx: 2200, epoch: 0, batch: 2200, avg loss: 3.398549, normalized loss: 2.253602, ppl: 29.920664, speed: 1.14 step/s

[2019-01-08 11:32:45,016 INFO train.py:550] step_idx: 2300, epoch: 0, batch: 2300, avg loss: 3.456284, normalized loss: 2.311337, ppl: 31.698973, speed: 1.14 step/s

[2019-01-08 11:34:12,223 INFO train.py:550] step_idx: 2400, epoch: 0, batch: 2400, avg loss: 3.363871, normalized loss: 2.218924, ppl: 28.900839, speed: 1.15 step/s

[2019-01-08 11:35:39,475 INFO train.py:550] step_idx: 2500, epoch: 0, batch: 2500, avg loss: 3.327908, normalized loss: 2.182961, ppl: 27.879944, speed: 1.15 step/s

[2019-01-08 11:37:07,678 INFO train.py:550] step_idx: 2600, epoch: 0, batch: 2600, avg loss: 3.421682, normalized loss: 2.276735, ppl: 30.620865, speed: 1.13 step/s

[2019-01-08 11:38:35,903 INFO train.py:550] step_idx: 2700, epoch: 0, batch: 2700, avg loss: 3.263523, normalized loss: 2.118576, ppl: 26.141462, speed: 1.13 step/s

[2019-01-08 11:40:03,281 INFO train.py:550] step_idx: 2800, epoch: 0, batch: 2800, avg loss: 3.216509, normalized loss: 2.071562, ppl: 24.940903, speed: 1.14 step/s

[2019-01-08 11:41:31,316 INFO train.py:550] step_idx: 2900, epoch: 0, batch: 2900, avg loss: 3.213425, normalized loss: 2.068478, ppl: 24.864092, speed: 1.14 step/s

[2019-01-08 11:42:58,778 INFO train.py:550] step_idx: 3000, epoch: 0, batch: 3000, avg loss: 3.235397, normalized loss: 2.090450, ppl: 25.416462, speed: 1.14 step/s

6.4 双卡原始数据

Event	Calls	Total	Min.	Max.	Ave.	Ratio.
all_reduce	1850	10724.6	0.045056	346.129	5.79709	0.287914
mul_grad	1920	5545.06	1.26874	19.9496	2.88805	0.148863
read	40	4044.45	0.002048	1045.94	101.111	0.108578
dropout	1240	3327.5	0.226304	7.69126	2.68347	0.0893304
mul	1920	2523.36	0.601088	4.81178	1.31425	0.0677423
matmul_grad	740	1609.28	0.63488	11.1002	2.1747	0.0432028
scale	8200	1205.09	0.008192	14.0677	0.146962	0.0323518
adam	3700	1074.63	0.009216	8.77158	0.29044	0.0288495
matmul	740	916.901	0.330752	8.17664	1.23906	0.0246152
elementwise_add_grad		1480	786.785	0.026624	19.3997	0.531612
0.0211221						
dropout_grad	1240	762.282	0.038912	129.66	0.614744	0.0204643

layer_norm_grad	640	615.1	0.567296	28.7887	0.961094	0.016513
sum	880	530.648	0.16384	129.839	0.603009	0.0142458
transpose2	1440	524.915	0.141312	287.307	0.364524	0.0140919
fill_zeros_like	2520	422.081	0.008192	8.8105	0.167493	0.0113312
transpose2_grad	1440	399.192	0.14336	7.92474	0.277217	0.0107167
elementwise_add	1480	363.149	0.033792	1.84218	0.245371	
0.00974914						
layer_norm	640	325.002	0.443392	0.60416	0.507816	0.00872504
softmax_grad	360	304.669	0.523264	71.3882	0.846302	0.00817916
elementwise_pow	20	290.75	0.125952	103.953	14.5375	0.00780551
softmax	360	237.429	0.596992	0.76288	0.659524	0.00637403
relu_grad	240	214.126	0.644096	7.12192	0.89219	0.00574843
lookup_table	80	175.104	0.10752	60.8594	2.1888	0.00470086
relu	240	121.154	0.442368	0.526336	0.50481	0.00325252
elementwise_mul	80	50.7177	0.008192	18.0654	0.633972	
0.00136157						
cast	20	27.5794	0.002048	26.9025	1.37897	0.000740399
softmax_with_cross_entropy	20	24.831	1.16531	1.27795	1.24155	
0.000666615						
fill_constant	80	21.1753	0.008192	10.7684	0.264691	0.000568474
lookup_table_grad	40	16.7782	0.257024	2.19853	0.419456	
0.00045043						
elementwise_min	20	16.5775	0.009216	16.342	0.828877	
0.000445042						
reshape2_grad	1460	13.652	0.002048	2.49651	0.00935066	
0.000366502						
softmax_with_cross_entropy_grad	20	11.1913	0.520192	0.571392	0.559565	
0.000300442						
label_smooth	20	8.76339	0.39936	0.502784	0.43817	0.000235263
one_hot	20	5.95251	0.1792	0.891904	0.297626	0.000159802
reshape2	1460	4.63789	0.002016	0.129024	0.00317664	
0.000124509						
reduce_sum	40	1.12432	0.01024	0.155648	0.028108	3.01836e-05
reduce_sum_grad	20	0.724992	0.01536	0.065536	0.0362496	
1.94632e-05						
create_double_buffer_reader	20	0.596992	0.002048	0.16384	0.0298496	
1.60269e-05						
increment	20	0.574464	0.002048	0.19968	0.0287232	1.54221e-05
elementwise_mul_grad	20	0.418816	0.01024	0.037888	0.0209408	
1.12436e-05						

elementwise_div	20	0.413696	0.01024	0.033792	0.0206848	1.11061e-05
elementwise_div_grad	20	0.4096	0.01024	0.033792	0.02048	1.09962e-05
ScopeBufferedSSAGraphExecutorAfterRun	10	0	0	0	0	0

[2019-01-08 11:00:48,989 INFO train.py:550] step_idx: 100, epoch: 0, batch: 100, avg loss: 6.566169, normalized loss: 5.421222, ppl: 710.642029, speed: 0.95 step/s

[2019-01-08 11:02:29,351 INFO train.py:550] step_idx: 200, epoch: 0, batch: 200, avg loss: 5.718982, normalized loss: 4.574035, ppl: 304.594757, speed: 1.00 step/s

[2019-01-08 11:04:09,340 INFO train.py:550] step_idx: 300, epoch: 0, batch: 300, avg loss: 5.585959, normalized loss: 4.441012, ppl: 266.655884, speed: 1.00 step/s

[2019-01-08 11:05:49,441 INFO train.py:550] step_idx: 400, epoch: 0, batch: 400, avg loss: 5.397211, normalized loss: 4.252264, ppl: 220.789795, speed: 1.00 step/s

[2019-01-08 11:07:29,135 INFO train.py:550] step_idx: 500, epoch: 0, batch: 500, avg loss: 4.806800, normalized loss: 3.661853, ppl: 122.339493, speed: 1.00 step/s

[2019-01-08 11:09:10,416 INFO train.py:550] step_idx: 600, epoch: 0, batch: 600, avg loss: 4.372459, normalized loss: 3.227512, ppl: 79.238235, speed: 0.99 step/s

[2019-01-08 11:10:50,502 INFO train.py:550] step_idx: 700, epoch: 0, batch: 700, avg loss: 4.256532, normalized loss: 3.111585, ppl: 70.564850, speed: 1.00 step/s

[2019-01-08 11:12:30,406 INFO train.py:550] step_idx: 800, epoch: 0, batch: 800, avg loss: 4.006369, normalized loss: 2.861422, ppl: 54.947002, speed: 1.00 step/s

[2019-01-08 11:14:10,850 INFO train.py:550] step_idx: 900, epoch: 0, batch: 900, avg loss: 4.117659, normalized loss: 2.972712, ppl: 61.415276, speed: 1.00 step/s

[2019-01-08 11:15:51,212 INFO train.py:550] step_idx: 1000, epoch: 0, batch: 1000, avg loss: 4.102848, normalized loss: 2.957901, ppl: 60.512356, speed: 1.00 step/s

[2019-01-08 11:17:30,805 INFO train.py:550] step_idx: 1100, epoch: 0, batch: 1100, avg loss: 4.012366, normalized loss: 2.867419, ppl: 55.277493, speed: 1.00 step/s

[2019-01-08 11:19:10,403 INFO train.py:550] step_idx: 1200, epoch: 0, batch: 1200, avg loss: 4.000741, normalized loss: 2.855795, ppl: 54.638649, speed: 1.00 step/s

[2019-01-08 11:20:50,357 INFO train.py:550] step_idx: 1300, epoch: 0, batch: 1300, avg loss: 3.951605, normalized loss: 2.806658, ppl: 52.018795, speed: 1.00 step/s

[2019-01-08 11:22:29,761 INFO train.py:550] step_idx: 1400, epoch: 0, batch: 1400, avg loss: 3.940898, normalized loss: 2.795951, ppl: 51.464779, speed: 1.01 step/s

[2019-01-08 11:24:09,694 INFO train.py:550] step_idx: 1500, epoch: 0, batch: 1500, avg loss: 3.740206, normalized loss: 2.595260, ppl: 42.106682, speed: 1.00 step/s

[2019-01-08 11:25:49,552 INFO train.py:550] step_idx: 1600, epoch: 0, batch: 1600, avg loss: 3.623741, normalized loss: 2.478794, ppl: 37.477497, speed: 1.00 step/s

[2019-01-08 11:27:30,624 INFO train.py:550] step_idx: 1700, epoch: 0, batch: 1700, avg loss:

3.485543, normalized loss: 2.340596, ppl: 32.640129, speed: 0.99 step/s
[2019-01-08 11:29:10,612 INFO train.py:550] step_idx: 1800, epoch: 0, batch: 1800, avg loss: 3.501552, normalized loss: 2.356605, ppl: 33.166874, speed: 1.00 step/s
[2019-01-08 11:30:50,584 INFO train.py:550] step_idx: 1900, epoch: 0, batch: 1900, avg loss: 3.535040, normalized loss: 2.390093, ppl: 34.296383, speed: 1.00 step/s
[2019-01-08 11:32:31,959 INFO train.py:550] step_idx: 2000, epoch: 0, batch: 2000, avg loss: 3.351989, normalized loss: 2.207042, ppl: 28.559469, speed: 0.99 step/s
[2019-01-08 11:34:12,032 INFO train.py:550] step_idx: 2100, epoch: 0, batch: 2100, avg loss: 3.235979, normalized loss: 2.091032, ppl: 25.431265, speed: 1.00 step/s
[2019-01-08 11:35:52,372 INFO train.py:550] step_idx: 2200, epoch: 0, batch: 2200, avg loss: 3.280305, normalized loss: 2.135358, ppl: 26.583878, speed: 1.00 step/s
[2019-01-08 11:37:32,559 INFO train.py:550] step_idx: 2300, epoch: 0, batch: 2300, avg loss: 3.233625, normalized loss: 2.088678, ppl: 25.371473, speed: 1.00 step/s
[2019-01-08 11:39:12,345 INFO train.py:550] step_idx: 2400, epoch: 0, batch: 2400, avg loss: 3.264428, normalized loss: 2.119481, ppl: 26.165131, speed: 1.00 step/s
[2019-01-08 11:40:52,577 INFO train.py:550] step_idx: 2500, epoch: 0, batch: 2500, avg loss: 3.167117, normalized loss: 2.022170, ppl: 23.738943, speed: 1.00 step/s
[2019-01-08 11:42:31,882 INFO train.py:550] step_idx: 2600, epoch: 0, batch: 2600, avg loss: 3.235634, normalized loss: 2.090687, ppl: 25.422474, speed: 1.01 step/s
[2019-01-08 11:44:12,647 INFO train.py:550] step_idx: 2700, epoch: 0, batch: 2700, avg loss: 3.076623, normalized loss: 1.931676, ppl: 21.685053, speed: 0.99 step/s
[2019-01-08 11:45:52,613 INFO train.py:550] step_idx: 2800, epoch: 0, batch: 2800, avg loss: 3.056093, normalized loss: 1.911147, ppl: 21.244402, speed: 1.00 step/s
[2019-01-08 11:47:32,949 INFO train.py:550] step_idx: 2900, epoch: 0, batch: 2900, avg loss: 3.018314, normalized loss: 1.873367, ppl: 20.456776, speed: 1.00 step/s
[2019-01-08 11:49:12,744 INFO train.py:550] step_idx: 3000, epoch: 0, batch: 3000, avg loss: 3.039754, normalized loss: 1.894807, ppl: 20.900110, speed: 1.00 step/s

七、XPU支持transformer训练需求

- 内存搬运op: transpose, reshape, lookup_table
- 计算op (包含前后向) : mul, matmul, elementwise, softmax, batch_norm等
- 片间通讯: all_reduce
- 其他: dropout

参考资料

1. <https://blog.csdn.net/pipisorry/article/details/84946653>
2. <https://blog.csdn.net/u014595019/article/details/52826423>

